利用C11模拟伪闭包实现连接的安全回收

Posted on 2023-04-10 18:39:10 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2OEQEc6p4k79cXsTr6dOVfZbo79)

简介

之前的异步服务器为echo模式，但其存在安全隐患，就是在极端情况下客户端关闭导致触发写和读回调函数，二者都进入错误处理逻辑，进而造成二次析构的问题。
下面我们介绍通过C11智能指针构造成一个伪闭包的状态延长session的生命周期。

智能指针管理Session

我们可以通过智能指针的方式管理Session类，将acceptor接收的链接保存在Session类型的智能指针里。由于智能指针会在引用计数为0时自动析构，所以为了防止其被自动回收，也方便Server管理Session，因为我们后期会做一些重连踢人等业务逻辑，我们在Server类中添加成员变量，该变量为一个map类型，key为Session的uid，value为该Session的智能指针。

```
1. class CServer
2. {
3. public:
4.     CServer(boost::asio::io_context& io_context, short port);
5.     void ClearSession(std::string);
6. private:
7.     void HandleAccept(shared_ptr<CSession>, const boost::system::error_code & error);
8.     void StartAccept();
9.     boost::asio::io_context &_io_context;
10.    short _port;
11.    tcp::acceptor _acceptor;
12.    std::map<std::string, shared_ptr<CSession>> _sessions;
13. };
```

通过Server中的_sessions这个map管理链接，可以增加Session智能指针的引用计数，只有当Session从这个map中移除后，Session才会被释放。
所以在接收连接的逻辑里将Session放入map

```
void CServer::StartAccept() {
    shared_ptr<CSession> new_session = make_shared<CSession>(_io_context, this);
    _acceptor.async_accept(new_session->GetSocket(), std::bind(&CServer::HandleAccept, this, new_session, placeholders::_1));
}

void CServer::HandleAccept(shared_ptr<CSession> new_session, const boost::system::error_code& error){
    if (!error) {
        new_session->Start();
        _sessions.insert(make_pair(new_session->GetUuid(), new_session));
    }
    else {
        cout << "session accept failed, error is " << error.what() << endl;
    }

    StartAccept();
}
```

StartAccept函数中虽然new_session是一个局部变量，但是我们通过bind操作，将new_session作为数值传递给bind函数，而bind函数返回的函数对象内部引用了该new_session所以引用计数增加1，这样保证了new_session不会被释放。
在HandleAccept函数里调用session的start函数监听对端收发数据，并将session放入map中，保证session不被自动释放。
此外，需要封装一个释放函数，将session从map中移除，当其引用计数为0则自动释放

```
void CServer::ClearSession(std::string uuid) {
    _sessions.erase(uuid);
}
```

Session的uuid

关于session的uuid可以通过boost提供的生成唯一id的函数获得，当然你也可以自己实现雪花算法。

```
CSession::CSession(boost::asio::io_context& io_context, CServer* server):
    _socket(io_context), _server(server){
    boost::uuids::uuid  a_uuid = boost::uuids::random_generator()();
    _uuid = boost::uuids::to_string(a_uuid);
}
```

另外我们修改Session中读写回调函数关于错误的处理，当读写出错的时候清除连接

```
void CSession::HandleWrite(const boost::system::error_code& error) {
    if (!error) {
        std::lock_guard<std::mutex> lock(_send_lock);
        _send_que.pop();
        if (!_send_que.empty()) {
            auto &msgnode = _send_que.front();
            boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_max_len),
                std::bind(&CSession::HandleWrite, this, std::placeholders::_1));
        }
    }
    else {
        std::cout << "handle write failed, error is " << error.what() << endl;
        _server->ClearSession(_uuid);
    }
}

void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferred){
    if (!error) {

        cout << "read data is " << _data << endl;
        //发送数据
        Send(_data, bytes_transferred);
        memset(_data, 0, MAX_LENGTH);
        _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2));
    }
    else {
        std::cout << "handle read failed, error is " << error.what() << endl;
        _server->ClearSession(_uuid);
    }
}
```

隐患

正常情况下上述服务器运行不会出现问题，但是当我们像上次一样模拟，在服务器要发送数据前打个断点，此时关闭客户端，在服务器就会先触发写回调函数的错误处理，再触发读回调函数的错误处理，这样session就会两次从map中移除，因为map中key唯一，所以第二次map判断没有session的key就不做移除操作了。
但是这么做还是会有崩溃问题，因为第一次在session写回调函数中移除session，session的引用计数就为0了，调用了session的析构函数，这样在触发session读回调函数时此时session的内存已经被回收了自然会出现崩溃的问题。解决这个问题可以利用智能指针引用计数和bind的特性，实现一个伪闭包的机制延长session的生命周期。

如何构造伪闭包

思路:
1　利用智能指针被复制或使用引用计数加一的原理保证内存不被回收
2　bind操作可以将值绑定在一个函数对象上生成新的函数对象，如果将智能指针作为参数绑定给函数对象，那么智能指针就以值的方式被新函数对象使用，那么智能指针的生命周期将和新生成的函数对象一致，从而达到延长生命的效果。
我们按照上面的思路改写我们的回调函数

```
1. void HandleRead(const boost::system::error_code& error,
2. size_t  bytes_transferred, shared_ptr<CSession> _self_shared);
3. void HandleWrite(const boost::system::error_code& error, shared_ptr<CSession> _self_share
   d);
```

以HandleWrite举例,在bind时传递_self_shared指针增加其引用计数，这样_self_shared的生命周期就和async_write的第二个参数(也就是asio要求的回调函数对象)生命周期一致了。

```
1. void CSession::HandleWrite(const boost::system::error_code& error, shared_ptr<CSession> _
   self_shared) {
2.     if (!error) {
3.         std::lock_guard<std::mutex> lock(_send_lock);
4.         _send_que.pop();
5.         if (!_send_que.empty()) {
6.             auto &msgnode = _send_que.front();
7.             boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_ma
   x_len),
8.                 std::bind(&CSession::HandleWrite, this, std::placeholders::_1, _self_shared));
9.         }
10.    }
11.    else {
12.        std::cout << "handle write failed, error is " << error.what() << endl;
13.        _server->ClearSession(_uuid);
14.    }
15. }
```

同样道理HandleRead内部也实现了类似的绑定

```
1. void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferre
   d, shared_ptr<CSession> _self_shared){
2.     if (!error) {
3.
4.         cout << "read data is " << _data << endl;
5.         //发送数据
6.         Send(_data, bytes_transferred);
7.         memset(_data, 0, MAX_LENGTH);
8.         _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSessio
   n::HandleRead, this,
9.             std::placeholders::_1, std::placeholders::_2, _self_shared));
10.    }
11.    else {
12.        std::cout << "handle read failed, error is " << error.what() << endl;
13.        _server->ClearSession(_uuid);
14.    }
15. }
```

除此之外，我们也要在第一次绑定读写回调函数的时候传入智能指针的值,但是要注意传入的方式，不能用两个智能指针管理同一块内存，如下用法是错误的。

```
1. void CSession::Start(){
2.     memset(_data, 0, MAX_LENGTH);
3.     _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSessio
   n::HandleRead, this,
4.         std::placeholders::_1, std::placeholders::_2, shared_ptr<CSession>(this)));
5. }
```

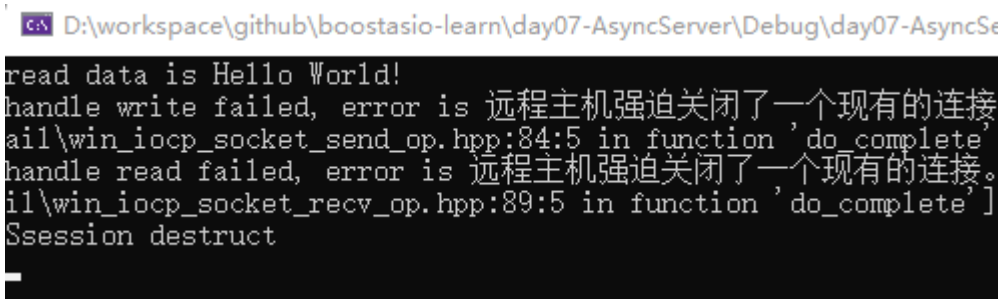
shared_ptr<CSession>(this)生成的新智能指针和this之前绑定的智能指针并不共享引用计数，所以要通过shared_from_this()函数返回智能指针，该智能指针和其他管理这块内存的智能指针共享引用计数。

```
1. void CSession::Start(){
2.     memset(_data, 0, MAX_LENGTH);
3.     _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSessio
   n::HandleRead, this,
4.         std::placeholders::_1, std::placeholders::_2, shared_from_this()));
5. }
```

shared_from_this()函数并不是session的成员函数，要使用这个函数需要继承std::enable_shared_from_this<Session>

```
1. class CSession:public std::enable_shared_from_this<CSession>
2. {
3. public:
4.     CSession(boost::asio::io_context& io_context, CServer* server);
5.     tcp::socket& GetSocket();
6.     std::string& GetUuid();
7.     void Start();
8.     void Send(char* msg,  int max_length);
9. private:
10.     void HandleRead(const boost::system::error_code& error, size_t  bytes_transferred, shared_ptr<CSession> _self_shared);
11.     void HandleWrite(const boost::system::error_code& error, shared_ptr<CSession> _self_shared);
12.     tcp::socket _socket;
13.     std::string _uuid;
14.     char _data[MAX_LENGTH];
15.     CServer* _server;
16.     std::queue<shared_ptr<MsgNode> > _send_que;
17.     std::mutex _send_lock;
18. };
```

同样的道理，我们在发送的时候也要绑定智能指针作为参数, 这里不做赘述。
再次测试，链接可以安全释放，并不存在二次释放的问题。可以在析构函数内打印析构的信息，发现只析构一次



总结

我们通过C11的bind和智能指针实现了类似于go，js等语言的闭包功能，保证在回调函数触发之前Session都是存活的。
源码链接
https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3612) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(570) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)